

HashCAT 精读笔记：面向 CAT 的不确定性削减哈希方法

原文：3726302.3730072.pdf (SIGIR '25, Ma et al., 安徽大学) 代码：https://github.com/sherklock/Intelligent-Education/tree/main/HashCAT-main (已 clone 到本目录 Intelligent-Education/HashCAT-main/HashCAT-main/)
笔记于 2026-06-27

一、解决什么问题

1.1 背景：CAT 的两个轮子

计算机自适应测试 (CAT) 的目标是用尽量少的题准确估出一个考生的能力。它靠两个模块交替转：

- **认知诊断模型 CDM**：根据考生已答的题，估出他当前能力 θ ；
- **习题选择模块 ESM**：根据当前 θ ，从题库里挑“最值得问的下一题”。

答一题 $\rightarrow$ 更新能力 $\rightarrow$ 再挑题 $\rightarrow$ 再答……循环到 T 步停止，最后那个能力估计就是结果。

1.2 痛点：现有选题策略「准」但「慢」

挑题这一步是研究热点，但现有两类方法都很慢：

- **基于统计的方法** (MFI 最大 Fisher 信息、KLI、MAAT、BECAT)：每一步要给题库里每道题算一个“信息量” (比如期望模型变化 EMC)，题库一大就要反复跑 CDM 前向，时间爆炸。论文表 2 里 BECAT 在 step=20 时每生选题要花 **1500 多秒**，MAAT 也要上百秒。
- **数据驱动方法** (NCAT、GMOCAT)：用强化学习训一个选题智能体，训练阶段 agent-environment 反复交互，训练成本极高 (表 3，GMOCAT 在 PTADISC 上训练要 1400 多秒)。

一句话痛点：效果 (AUC/ACC) 和效率 (时间) 难以兼得。

1.3 本文思路：把“选题”重构成“检索”，用哈希加速

作者的核心洞察：**选题本质是“在考生和题之间做匹配”，那就是一个检索问题**。检索领域加速的看家本领是 **学习哈希 (Learning to Hash)** —— 把对象编码成紧凑的 0/1 二值码，匹配时用位运算 (异或、汉明距离)，比浮点内积快几个数量级。

于是 HashCAT 把考生能力、习题难度都编成二值哈希码 b_s, b_e ，选题时直接对预存的哈希码做位运算，避免实时跑 CDM。结果 (表 2)：每生选题降到 **0.005~0.09 秒**，比统计方法快几个数量级，AUC 还领先大多数基线。

1.4 三个挑战 $\rightarrow$ 三个模块

把哈希塞进 CAT 不是直接套，作者点出三个挑战，分别对应三个设计：

挑战	设计
哈希码怎么生成、且有“物理意义” (对应知识概念掌握/难度)？	HRGM: VAE + Gumbel-Softmax 生成二值码
实值 $\rightarrow$ 哈希有量化损失，怎么保住信息？	IAM: 用 LSH 做一致性约束
二值码上的内积/余弦抓不住师生交互，诊断函数怎么设计？	HDM: XOR + 内积 的认知诊断函数 (带单调性证明)
怎么用哈希码设计可靠又快的选题策略？	不确定性削减 + ϵ -greedy/UCB 混合

二、问题定义 (输入 / 输出)

符号：- 考生集 S ，题库 E ，知识概念集 C ($|C|$ 是概念数，比如 NIPS-2020 是 56)。- 作答日志三元组 $R_s(s_i, e_j, r_{ij})$, $r_{ij} \in \{0, 1\}$ 表示答对/错。- 假设考生真实能力在一次测试中不变。

哈希表征生成阶段 (离线训练)：- 输入：作答日志 R 。- 输出：考生能力哈希码 $b_s \in \{0, 1\}^{|C|}$ 、习题难度哈希码 $b_e \in \{0, 1\}^{|C|}$ ，外加习题的实值难度向量 δ 、区分度 ψ 。

CAT 选题阶段 (在线): - 输入: 考生当前能力 (由基础 CDM 给的 θ , 再过同一个哈希生成器变成 b_s)、候选题库每道题的预存 $\{b_e, \delta, \psi\}$ 。- 输出: 下一道要问的题 e^t 。

一个具体小例子 (取 $|C| = 5$ 个概念, 便于手算):

假设某考生能力哈希码 $b_s = [1, 1, 0, 1, 0]$ (在概念 1、2、4 上“掌握”), 某习题难度哈希码 $b_e = [1, 0, 0, 1, 0]$ (考概念 1、4)。后面所有公式我们都用这对向量走数。

三、模型结构 (数据怎么流)

整体四块, 对应代码 CAT/model/HashModel.py 的 BinaryCode 类和 CAT/strategy/HashMABUCB_strategy.py:

```

作答记录 R
| Eq(2): one-hot x 可训练矩阵 -> 嵌入 h_s, h_e, h_K
v
知识掌握向量 gamma / 难度向量 delta (实值, R^{|C|})
| HRGM: VAE 编码 -> 采样 z -> Gumbel-Softmax 二值化 Eq(3)
v
哈希码 b_s, b_e in {0,1}^{|C|} --IAM(LSH一致性 Eq7)--> 拉近实值与哈希的邻居结构
| HDM: 诊断函数 F_H Eq(8) -> 预测答对概率 y -> 交叉熵 Eq(11) 反传
v
(离线训练完, 把每题的 b_e, delta, psi 存成 JSON 题库)
|
v 在线选题: 不确定性削减 dH = H_prior - H_post Eq(13-16)
      前 S 步 epsilon-greedy, 后面 UCB Eq(17)
v
选出一题
下面逐块讲。

```

3.1 HRGM: 怎么从实值变出二值码 (Eq 2、3)

第一步 Eq(2): 把考生/习题/概念 one-hot 经可训练矩阵 W_s, W_e, W_k 变成 d 维嵌入 (代码里 low_dim), 再用考生嵌入 $\times$ 概念嵌入转置得到 $|C|$ 维的“概念掌握向量” γ 、习题“概念难度向量” δ , 区分度 $\psi = \sigma(h_e W + b)$ 。

对应代码 (HashModel.py 的 forward):

```
batch_question_emb = torch.sigmoid(torch.mm(batch_question_emb, knowledge_low_emb.T)) # = delta
```

第二步 Eq(3): 把实值 δ 喂 VAE, 编码出均值 μ 、标准差 σ , 重参数采样 $z_e = \mu + \varepsilon\sigma$, 再用 **Gumbel-Softmax** 把连续 z 推成 0/1, 且训练时可微、推理时硬离散。代码:

```
question_latent, ... = self.question_vae(batch_question_emb) # z_e = mu+eps*sigma
question_binary_code = self.binary_gumbel_softmax(question_latent) # -> {0,1}^{|C|}
```

binary_gumbel_softmax 把每个 logit 拼上一个 0, 做 F.gumbel_softmax(..., hard=True), 取第 0 维当作该位取 1 的概率——这正是 Eq(3) 的实现。 τ (代码 self.tau=100) 控制平滑度。

VAE 的损失 Eq(4) = 重构损失 $\|\delta - \hat{\delta}\|^2 + \text{KL 散度 } D_{KL}(q(z|x)||p(z))$, 让潜变量贴近标准正态、保证泛化。代码里 vae_loss 就是 student/question 两侧的 mse + KL 之和。

3.2 IAM: 别让量化把信息丢光 (Eq 5、6、7)

实值连续 $\rightarrow$ 二值离散, 天然有量化损失。IAM 的办法: 用 **LSH** 分别给实值表征和哈希表征分桶, 要求两套桶结构一致。

- 实值哈希函数 Eq(5): $h_e = \lfloor (z_e \cdot a + c)/w \rfloor$, 随机投影后量化分桶 (代码 LSH_Hash_Real)。
- 哈希码的哈希 Eq: 从 $|C|$ 维里随机抽 d 维当桶标识 (代码 LSH_Hash_Binary 用 index_select)。
- 相似度矩阵 Eq(6): 同桶 $A_{ij} = 1$ 否则 0 (代码 Build_LSH_Similarity_Matrix)。
- 一致性损失 Eq(7): 在哈希相似 (A') 的对上压低实值欧氏距离; 在实值相似 (A) 的对上抬高哈希内积。代码 LSH_Consistency_Loss:

```
real_loss =  $\sum \|z_i - z_j\|^2 \cdot A\_binary$  # 哈希相似 → 实值也要近
binary_loss =  $\sum (b_i \cdot b_j) \cdot A\_real$  # 实值相似 → 哈希内积要大
```

这样实值和哈希两个空间的“谁和谁像”被对齐，量化没把邻居关系搞乱。

3.3 HDM：二值码上的诊断函数（重点，见第四节 Eq 8-11）

3.4 选题：不确定性削减（重点，见第四节 Eq 13-17）

四、关键公式详解（带数值演算）

沿用例子 $b_s = [1, 1, 0, 1, 0]$ 、 $b_e = [1, 0, 0, 1, 0]$ 。

公式 8：基于哈希的认知诊断函数

$$F_H(b_s, b_e) = Q \cdot [\alpha(1 - b_s \oplus b_e) + \beta \text{Interaction}(b_s, b_e)] \cdot \psi$$

- 符号： $\oplus$ 是逐位异或； $1 - b_s \oplus b_e$ 衡量“考生掌握和题目要求的吻合度”（异或为 0 即一致 $\rightarrow 1-0=1$ 表示这位匹配）；Interaction 是内积 $b_s \cdot b_e$ ，抓“共同为 1”的对齐； ψ 是区分度； Q 是 Q-矩阵； α, β 默认 0.5、0.7。
- 数值走一遍（论文里这两项实现成“按位取均值”，见代码）：
 - 异或： $b_s \oplus b_e = [0, 1, 0, 0, 0]$ ， $XOR_Sum = 1$ ，按 5 维取均值 = 0.2，所以 $1 - \overline{b_s \oplus b_e} = 0.8$ （匹配度高，因为 5 位里只有 1 位不同）。
 - 内积/匹配：逐位相乘 $[1, 0, 0, 1, 0]$ ，和 = 2，均值 = 0.4。
 - 取 $\psi = 2$ （区分度），则括号内 = $0.5 \times 0.8 + 0.7 \times 0.4 = 0.4 + 0.28 = 0.68$ ，乘 ψ 得 1.36，再过 Q-矩阵和后面的预测网络 sigmoid $\rightarrow$ 答对概率。
- 代码对应 (HashModel.py)：

```
ability_difference = STE_XOR.apply(student_binary_code, question_binary_code).mean(1) # b_s XOR b_e 的均值
matching = (student_binary_code * question_binary_code).mean(1) # 内积均值
predicted_ability = e_discrimination * (alpha*(1-ability_difference) + beta*matching) * kn_emb
```

注意作者用了自定义 STE_XOR（直通估计器）让不可导的 XOR 能反传梯度。- 作用：这是“哈希版的 NeuralCDM”，替代了普通哈希里抓不住师生交互的内积/余弦。它还配了 NoneNegClipper（把预测网络权重夹成非负），把单调性假设落到实处。

公式 9、10：单调性证明（为什么这个诊断函数合理）

教育诊断有个铁律——**单调性假设**：考生能力越高，答对概率不能反而下降。作者证 $f(b_s) = \alpha(1 - b_s \oplus b_e) + \beta(b_s \cdot b_e)$ 对 b_s 单调不减：

$$\frac{\partial f}{\partial b_s} = \alpha \frac{\partial(1 - b_s \oplus b_e)}{\partial b_s} + \beta b_e \geq 0$$

直观理解： b_s 某位从 0 变 1（能力变强），要么让该位和 b_e 更一致（异或项变小 $\rightarrow 1 - \oplus$ 变大），要么让内积变大，两项都不会让结果变小。 $\alpha, \beta > 0$ 保证整体单调增。作用：保证模型符合教育学常识，能力强的人估出来答对率更高，结果可信。

公式 12：训练总损失

$$\mathcal{L}_H = \mathcal{L}_{Hcdm} + \mathcal{L}_S^H + \mathcal{L}_E^H + \mathcal{L}_{con}$$

= 诊断交叉熵 (Eq 11, 预测对错) + 考生侧 VAE 损失 + 习题侧 VAE 损失 (Eq 4) + LSH 一致性损失 (Eq 7)。四项一起把“二值码生成 + 信息保真 + 诊断准确”联合优化。

公式 14、15：先验信息熵（选题前的不确定性）

$$P_c = \left\{ 1 + \exp\{-\kappa \cdot (M_{se}(b_s, b_e) \cdot \psi) + \zeta \cdot \mathbb{E}(\delta)\} \right\}^{-1}$$

$$H_{prior} = -P_c \log P_c - (1 - P_c) \log(1 - P_c)$$

- 符号: $M_{se} = 1 - \text{汉明距离}$ (考生与题的哈希匹配度), ψ 区分度, $\mathbb{E}(\delta)$ 题目平均难度, κ, ζ 缩放参数 (论文调出 $\kappa = 0.001, \zeta = 10$). H_{prior} 是把答对概率 P_c 套二项熵—— P_c 越接近 0.5, 熵越大 (最不确定), 越接近 0 或 1 熵越小。
- 数值: 汉明距离 = 异或和/维数 = $1/5 = 0.2$, 故 $M_{se} = 0.8$. 代码里 $P_correct = 1/(1+\exp(-\text{theta}*\text{matching}*\text{discrim} + \text{tbeta}*\text{difficulty}))$. 若简单取指数内 = 0, 则 $P_c = 0.5$, $H_{prior} = -0.5 \log 0.5 - 0.5 \log 0.5 = \log 2 \approx 0.693$ (最大熵, 说明这题对该生信息量潜力最大)。
- 作用: 量化“问这题之前, 我对这考生能力有多没底”。
- 代码对应 (HashMABUCB_strategy.py):

```
hamming_distance = np.mean(student_emb_binary ^ filtered_question_binaries, axis=1)
matching = 1 - hamming_distance
P_correct = 1/(1+np.exp(-theta*matching*discrim + tbeta*difficulty))
H_prior = -P_correct*np.log(P_correct+1e-5) - (1-P_correct)*np.log(1-P_correct+1e-5)
```

注意 $\wedge$ 就是 numpy 的逐位异或, 整个打分纯位运算 + 向量化, 这是它快的根源。

公式 16: 后验信息熵 + 公式 13: 信息增益

$$H_{post} = H_{prior} \cdot \left(-\frac{\delta_{gap}}{\exp(\mathbb{E}(\delta))} \right), \quad \delta_{gap} = |\mathbb{E}(\theta) - \mathbb{E}(\delta)|$$

$$\Delta H = H_{prior} - H_{post}$$

- 思想: 假设答完这题, 不确定性会降。 δ_{gap} 是“考生能力均值与题目难度均值的差距”——题和考生能力越贴 (gap 小), 后验熵压得越, 越能稳稳地确认能力; 用 $\exp(\mathbb{E}(\delta))$ 做难度衰减。
- 数值: 设 $\mathbb{E}(\theta) = 0.6$, $\mathbb{E}(\delta) = 0.5$, 则 $\delta_{gap} = 0.1$, $\exp(0.5) \approx 1.649$, $H_{post} = 0.693 \times (-0.1/1.649) = 0.693 \times (-0.0606) \approx -0.042$, $\Delta H = 0.693 - (-0.042) \approx 0.735$. ΔH 越大表示这题能削掉越多不确定性, 就越该选它。
- 作用: 把“选哪道题”变成“选 ΔH 最大的题”, 这是选题准则的核心。代码 $IG = H_prior - H_post$ 。

公式 17: ϵ -greedy + UCB 混合 (探索与利用)

$$UCB = \frac{R_e}{N + \omega} + \sqrt{\frac{2 \ln(T_Q)}{N + \omega}}$$

CAT 是马尔可夫过程, 光贪心选 ΔH 最大容易陷在局部。作者: 前 S 步用 ϵ -greedy (以 ϵ 概率随机探索, 否则选 ΔH 最大), S 步之后用 UCB。

- 符号: R_e 该题累计 ΔH 奖励, N 被选次数, T_Q 总选择次数, ω 防除零。第一项“利用” (历史表现好的题), 第二项“探索” (选得少的题这项大, 鼓励试)。
- 数值: 某题 $R_e = 2.0$, $N = 4$, $T_Q = 50$, $\omega \approx 10^{-9}$: 利用项 = $2.0/4 = 0.5$, 探索项 = $\sqrt{2 \ln 50/4} = \sqrt{2 \times 3.91/4} = \sqrt{1.956} \approx 1.40$, $UCB \approx 1.90$ 。一道刚被选过 1 次的题探索项会更大, 于是有机会被翻牌。
- 代码对应:

```
if it <= S: # 前 S 步 epsilon-greedy
    if random.random() < epsilon: best = random.choice(...)
    else: best = argmax(IG)
else: # 之后 UCB
    ucb = rewards/(counts+1e-9) + sqrt(2*log(total+1)/(counts+1e-9))
    best = argmax(ucb)
```

(代码里 $S=22$, $\text{theta}=0.001$, $\text{tbeta}=8$, 是对应数据集调好的值。)

五、代码实现对照 (速查表)

仓库结构: Intelligent-Education/HashCAT-main/HashCAT-main/

论文内容	代码位置
HRGM: VAE + Gumbel 二值化 (Eq 3)	CAT/model/HashModel.py → VAE、 binary_gumbel_softmax
HDM 诊断函数 F_H (Eq 8)	HashModel.py → forward 里 predicted_ability 那几行
XOR 直通估计器 (让异或可导)	HashModel.py → STE_XOR
单调性约束 (非负权重夹紧)	HashModel.py → NoneNegClipper + apply_clipper
IAM: LSH 一致性 (Eq 5-7)	HashModel.py → LSH_Hash_Real/Binary、 consistency_loss
不确定性削减选题 (Eq 13-17)	CAT/strategy/HashMABUCB_strategy.py → adapttest_select
题库哈希码预存	question_binary.py 生成 QuestionBinaryBank/*.json
训练入口	run_binary_module.py (训哈希)、run_cdm.py (训基础 CDM)、test_CAT.py (跑评测)
基线策略	CAT/strategy/ 下 MFI/KLI/MAAT/BECAT/BOBCAT_strategy.py

一个关键工程细节：**选题时不再跑神经网络**，而是直接读预存 JSON 里每题的 `binary_code / discrimination / difficulty`，对未答题做向量化的 $\hat{\cdot}$ (异或) + sigmoid + 熵计算。这就是它每生选题只要毫秒级、远快于要反复前向 CDM 的统计方法的根本原因。

六、小结

结论要点： - 在 NIPS-2020/JUNYI/ASSIST1213/PTADISC 四个数据集上，HashCAT 的 AUC/ACC 超过大多数基线，选题时延仅次于纯随机 (统计方法的 $1/1000 \sim 1/100000$)，训练时间也比 NCAT/GMOCAT 短 (表 3)。- 消融 (表 4)：去掉不确定性削减 (w/o-UR) 掉得最狠 (光靠汉明距离选不行)，IAM 和混合策略也都有贡献。

优点： 1. 把 CAT 选题重构为哈希检索，用位运算实现亚线性、毫秒级选题——真正同时拿下效率和效果。2. HDM 诊断函数带单调性证明，比传统哈希的内积更贴教育语义、更可解释。3. IAM 用 LSH 一致性缓解量化损失，是这类“二值化掉信息”问题的通用思路。

局限 / 可深挖： 1. 哈希码维度被绑定成 $|C|$ ，能力刻画的粒度受概念数限制； κ, ζ, τ, S 等超参对不同数据集需要重调 (论文 RQ4 也显示敏感)。2. 假设考生能力测试中不变 (标准 CAT 假设)，不适用于学习/遗忘动态——这点和读者关注的 KT 正好互补：KT 建模能力随时间变化，CAT 假设其静止。3. H_{post} 用的是启发式的指数衰减规则 (Eq 16) 而非真正重新诊断后的熵，是个近似，理论上还能更严谨。

对 KT/CAT 研究者的启发： 哈希/检索加速是 CAT 效率方向一条少有人走但很实在的路；其“实值 CDM → 二值码 → 位运算选题”的范式，也可迁移到大规模题库的冷启动选题、或 KT 里的快速相似题检索。